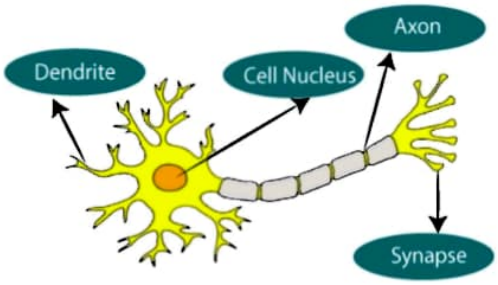
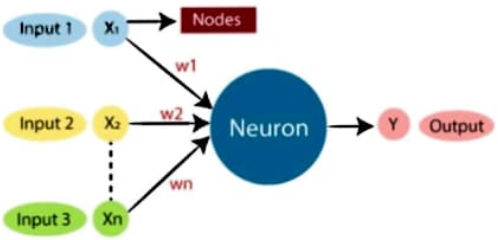
Neural Network

- The term "**Artificial Neural Network**" or "**Neural Networks**" or is derived from Biological neural networks that develop the structure of a human brain.
- Similar to the human brain that has neurons interconnected to one another, artificial neural networks also have neurons that are interconnected to one another in various layers of the networks.
- These neurons are known as nodes.
- An **Artificial Neural Network** in the field of **Artificial intelligence** where it attempts to mimic the network of neurons makes up a human brain so that computers will have an option to understand things and make decisions in a human-like manner



The given figure illustrates the typical diagram of Biological Neural Network.

The typical Artificial Neural Network looks something like the given figure.



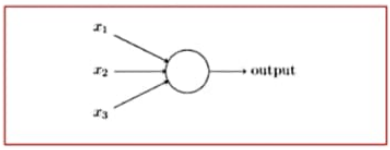
Neural network is a set of neurons organized in layers. Each neuron is a mathematical operation that takes it's input, multiplies it by it's weights and then passes the sum through the activation function to the other neurons. Neural network is learning how to classify an input through adjusting it's weights based on previous examples.

What is an artificial neuron?

An artificial neuron is a mathematical function based on a model of biological neurons, where each neuron takes inputs, weighs them separately, sums them up and passes this sum through a nonlinear function to produce output.

Perceptron: Building Block of Artificial Neural Network

- Perceptrons were developed in the 1950s and 1960s by the scientist Frank Rosenblatt,
- perceptron takes several binary inputs, *x*1, *x*2, . . ., and produces a single binary output



- Rosenblatt proposed a simple rule to compute the output. He introduced *weights*, *w*1,*w*2, . . ., real numbers expressing the importance of the respective inputs to the output.
- The neuron's output, 0 or 1, is determined by whether the weighted sum $\sum_j w_j x_j$ is less than or greater than some *threshold value*.
- Just like the weights, the threshold is a real number which is a parameter of the neuron.

$$\text{output} = \begin{cases} 0 & \text{if } \sum_j w_j x_j \leq \text{threshold} \\ 1 & \text{if } \sum_j w_j x_j > \text{threshold} \end{cases}$$

How does it work?

- The perceptron works on these simple steps

1. All the inputs *x* are multiplied with their weights *w*
2. *Add* all the multiplied values and call them *Weighted Sum.*
3. *Apply* that weighted sum to the correct *Activation Function*. For Example: Unit Step Activation Function.

The original **Perceptron** was designed to take a number of **binary** inputs, and produce one **binary** output (0 or 1).
The idea was to use different **weights** to represent the importance of each **input**, and that the sum of the values should be greater than a **threshold** value before making a decision like **true** or **false** (0 or 1).

✓**Why do we need Weights and Bias?**

Weights shows the strength of the particular node.

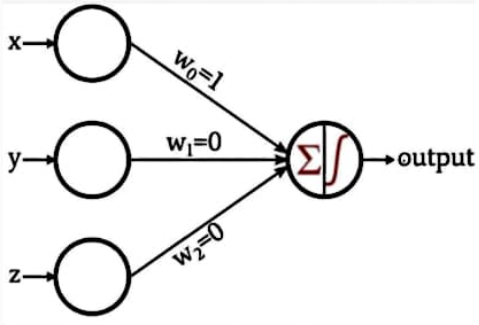
A bias value allows you to shift the activation function curve up or down.

✓**Why do we need Activation Function?**

In short, the activation functions are used to map the input between the required values like (0, 1) or (-1, 1).

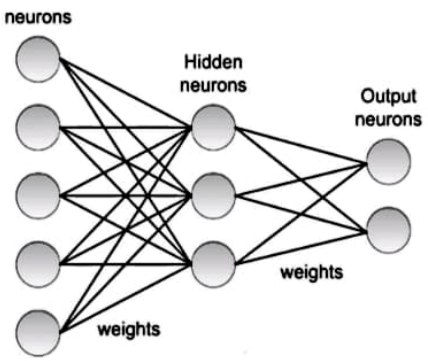
Single layer perceptrons

- A single-layer perceptron is the basic unit of a neural network. A perceptron consists of input values, weights and a bias, a weighted sum and activation function.



Multi Layer Perceptrons(MLPs)

- A Multilayer Perceptron has input and output layers, and one or more **hidden layers** with many neurons stacked together.

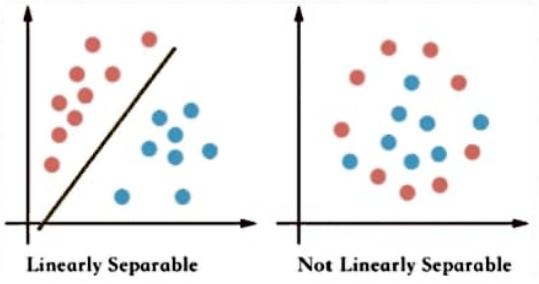


Activation Functions

- Activation functions refer to the functions used in neural networks to compute the weighted sum of input and biases, which is used to choose the neuron that can be fire or not.
- **An Activation Function** decides whether a neuron should be activated or not. This means that it will decide whether the neuron's input to the network is important or not in the process of prediction using simpler mathematical operations.
- The role of the Activation Function is to derive output from a set of input values fed to a node (or a layer).

Linearly Separable

- Linear separability is the concept wherein the separation of input space into regions is based on whether the network response is positive or negative
- Two sets of points (or classes) are called linearly separable, if at least one straight line(decision line) in the plane exists so that **all the points of one class are on one side of the line and all the points of the other class are on the other side** they are called linearly separable



If there exists weights(with bias) for which training input vectors have positive (correct) response, lie on one side of the decision boundary and negative (incorrect) response, lie on the other side of the decision boundary, then the problem is "linearly separable."

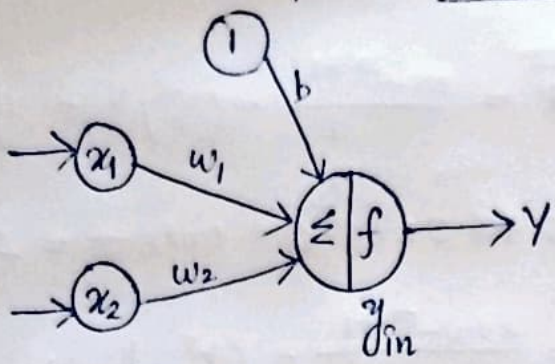
Representational power of perceptron

- A single perceptron can be used to represent many boolean functions.
- Perceptron can be used to represent functions which are linearly separable.
- For example, if we assume boolean values of 1 (true) and -1 (false), then one way to use a two-input perceptron to implement the AND function is to set the weights $wo = -3$, and $wl = wz = .5$.
- This perceptron can be made to represent the OR function instead by altering the threshold to $wo = -.3$.
- Perceptron can represent all of the primitive boolean functions AND, OR, NAND (1 AND), and NOR (1 OR). (linearly separable)
- Unfortunately, however, some boolean functions cannot be represented by a single perceptron, such as the XOR function.

XOR problem

- XOR is not linearly separable
- XOR cannot be represented by a single layer perceptron, but by MLP

Realization of AND Gate using Single layer Perceptron



$$y_{in} = b + \Sigma x_i w_i$$

$$Y = f(y_{in})$$

Activation function

$$f(x) = \begin{cases} 1, & \text{if } y_{in} > \theta \\ 0, & \text{if } y_{in} = \theta \\ -1, & \text{if } y_{in} < \theta \end{cases}$$

$$w_i(new) = w_i(old) + \underbrace{\alpha t x_i}_{\Delta w}$$

$$b_i(new) = b_i(old) + \underbrace{\alpha t}_{\Delta b}$$

Consider the weights and bias; $w_1 = 0$, $w_2 = 0$, $b = 0$ and learning rate $\alpha = 1$ and threshold $\theta = 0$

Epoch-1

x_1	x_2	b	t	y_{in}	Y	Δw_1 ($\alpha t x_1$)	Δw_2 ($\alpha t x_2$)	Δb (αt)	w_1 new	w_2 new	b new
1	1	1	1	0	0	1	1	1	1	1	1
1	-1	1	-1	1	1	-1	1	-1	0	2	0
-1	1	1	-1	2	1	1	-1	-1	1	1	-1
-1	-1	1	-1	-3	-1	0	0	0	1	1	-1